

Miller-Rabin primality test

Killian O'Brien

6G6Z0024 Applied Cryptography 2025/26

Lecture Week 11 – Mon 08 December 2025

- Definitively testing large integers for primeness is computationally hard (i.e. long) for large primes.
- But having large primes or *pseudo-primes* is important for many cryptography applications.
- Thankfully there are quicker *probabilistic* tests for primeness available.
 - such tests identify *pseudo-primes*, i.e. integers that share many properties of prime numbers or that are likely, to a high probability, to be prime.

Background

- Recall Fermat's Little Theorem: If p is a prime, and a an integer coprime to p , then

$$a^{p-1} \equiv 1 \pmod{p}.$$

- This can be turned into a test for primality by efficiently searching for the breakdown of this condition when the modulus is not in fact prime.
- So if we find an a that is coprime to n but

$$a^{n-1} \not\equiv 1 \pmod{n}$$

then we can conclude that n is not prime.

- Let n be an odd integer. Then we can express n as

$$n = 2^k \cdot q + 1,$$

for suitable integers k, q , where q is odd and $k \geq 1$.

- Note that these values of k and q can be quickly found, even for large n .
- Now, if n is prime, and a is an integer $1 < a < n - 1$, then Fermat's Little Theorem implies that

$$a^{n-1} \equiv a^{2^k q} \equiv 1 \pmod{n}.$$

- Consider the sequence of powers leading up to $a^{2^k q}$, namely,

$$a^q, a^{2q}, a^{2^2 q}, a^{2^3 q}, \dots, a^{2^k q},$$

and note that each element on this list is the square of the previous element.

- We know that the last element $a^{2^k q}$ is $\equiv 1 \pmod{n}$ if n is prime. So this means that somewhere earlier in the list the element must be $\equiv +1$ or $-1 \pmod{n}$, since ± 1 are the only two elements that square to 1.
- So we will know for certain that n is **not prime** if we encounter an element a , satisfying $1 \leq a \leq n - 1$ where the list of residues modulo n

$$a^q, a^{2q}, a^{2^2 q}, a^{2^3 q}, \dots, a^{2^{k-1} q},$$

does not contain any member congruent to 1 or -1 modulo n .

The algorithm

- Given an odd integer n , determine k and q such that

$$n = 2^k q + 1,$$

where q is odd.

- Select a random a , satisfying $1 < a < n - 1$
- Proceed through the list

$$a^q, a^{2q}, a^{2^2q}, a^{2^3q}, \dots, a^{2^{(k-1)}q} \pmod{n},$$

by starting with a^q and repeatedly squaring.

- If the list **does not** contain any element $\equiv \pm 1 \pmod{n}$ then stop and return **n is composite**.
- Otherwise return **inconclusive**, i.e. n may or may not be prime.

Repeated use of the algorithm

- It can be shown that the probability that if n is composite, it passes this test, i.e. returns **inconclusive**, is approximately $1/4 = 25\%$.
- So if we test t different values of a , the probability that a composite n will pass the test is approx. $(1/4)^t$.
- This means we can be very confident in the primeness of n . For example, a composite n passing 10 applications of this test can happen only with a probability less than $10^{-6} = 1/(1,000,000)$.

- The Miller-Rabin test is discussed in detail in [Section 2.6 in Stallings](#).
- On page 67 there is a pseudo-code definition for a function `Test(n)` implementing the test.
- Unfortunately the typesetting is not correct there. I think this is a result of the conversion to a pdf ebook perhaps?
- Here is a corrected version of that pseudo-code. Not that the if a `return` command executes, the function `Test` will cease executing and pass control back to whatever code or user invoked the function.

`Test(n)`

```
1. Find integers k, q with k > 0, q odd, so that n - 1 = 2^k * q;  
2. Select a random integer a, satisfying 1 < a < n - 1;  
3. if a^q mod n = 1 then return('inconclusive');  
4. for j = 0 to k - 1 do  
    5. if a^(2^j) * q mod n = n - 1 then return('inconclusive');  
6. return('composite')
```